

Interesting Learnings from Outages (Real-World Engineering Challenges #10)

A DNS mystery at Adevinta; A failover causing an outage at GitHub; The challenge whether to do a rollback at Reddit; and the difference between public and internal postmortems.



GERGELY OROSZ

JUL 18, 2023



150



3

Share



👋 Hi, this is [Gergely](#) with the monthly, free issue of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at Big Tech and startups through the lens of engineering managers and senior engineers.

If you're not a subscriber yet, you missed the issue on [What a senior software engineer is at Big Tech](#) and [a few others](#). Subscribe to get two full issues every week. Many subscribers expense this newsletter to their learning and development budget. If you have such a budget, here's [an email you could send to your manager](#). 📌

'Real-world engineering challenges' is [a series](#) in which I interpret interesting software engineering or engineering management case studies from tech companies. You might learn something new in these articles, as we dive into the concepts they contain.

An outage refers to an event when a system, service or application your team or company owns becomes unavailable or stops functioning as expected. The impact of the outage could range from very small (something that few users notice) to critical (when it impacts most customers). When an outage occurs, the number one priority is always to mitigate it. Once mitigation happens, it's time to investigate more thoroughly: [do an incident review, write a postmortem](#), and improve the resilience of the system.

As an engineer, it's never a good day to have an outage. And it's always cheaper to learn about the outages of others than to learn from our own one. Luckily, many tech

companies publish public postmortems following widespread outages. Today, we look through a couple of these incident reviews with the aim of learning from them. Perhaps some of these published case studies could help you make your systems more resilient.

We cover:

1. **Internal vs. public postmortems.** The difference between internal, customer-only and public postmortems.
2. **Adevinta: a DNS murder mystery.** A non-deterministic outage kept evading attempts to pinpoint it. When you are out of ideas on what could go wrong, it's usually caching issue or a DNS (domain name service) one. So was it DNS? Caching? Or both?
3. **GitHub: how investing in reliability can have bumps along the way.** Testing a failover caused a brief outage. Is this grounds for doing more or fewer failovers?
4. **Reddit: the difficult decision to try and fix on the spot, or to do a lengthy restore.** On 3/14 (Pi-day!), Reddit was down for 314 minutes. The long downtime was thanks to a Kubernetes cluster upgrade that went wrong, and where the root cause was very difficult to pinpoint.

1. Internal vs. public postmortems

It is *not* a given that companies publish incident reviews in public. In fact, most outages that happen tend to not be followed by a public postmortem at the majority of tech companies. This does not mean that there is no internal investigation and follow-up; it just means that most companies don't publish this document.

Internal postmortems are written *only* to be used within the company. This means that the language used can be more loose, details that are confidential can safely be shared, and this version of the document tends to be the most detailed version of an incident summary.

Internal postmortems typically go through an internal incident review. The higher the impact of the outage, the harder the questions usually asked on this review. The goal of

the incident review process is to understand what led to the outage; make changes on how to avoid similar situations; and capture and circulate learnings across the organization.

Customer-only postmortems are written to only be forwarded to customers, not to be shared with the public. These postmortems are often marked as “private and confidential,” reminding customers to not share the information publicly.

Customer-only postmortems tend to be more strictly reviewed to make sure no confidential information is published. Some companies will omit details that they don’t want to share with their customers. These details could be implementation details that the company doesn’t consider to be relevant for customers to know. They could also be relevant details that the company decides customers should not know about – or at least not all customers.

The more transparent the culture of a company, the more similar the internal postmortem will be to the customer-only postmortem. At startups and smaller companies, it’s common enough to share a copy of the internal postmortem directly with impacted customers.

As a large customer of a service, it is reasonable to ask for a postmortem. Often, this is called a **RCA** – root cause analysis. For example, when I worked at Uber on the payments team, and one of our payment providers would have an outage, we would request an RCA from that vendor.

Public-facing postmortems are ones published on the website of the vendor/tech company. These are available for anyone to read, not just customers. These postmortems tend to be thoroughly edited, with confidential information and jargon removed, so that the final document can speak to a broad audience. The following groups will read this incident review:

- **Customers.** The most important group: the postmortem will, hopefully, restore their trust in the company.
- **Techies.** Public postmortems are frequently circulated on tech forums. Software engineers, site reliability engineers (SREs) and other people working in tech who

are not customers of the company will read this document with the goal of learning from the mistakes of the company. In an indirect way, a company that regularly publishes high-quality incident reviews could attract tech folks interested in reliability.

- **Non-tech people.** As the postmortem is in the public arena, anyone else can read it: investors, regulators, and other interested parties.
- **The press.** Tech publications frequently analyze these public postmortems. The better-known the company, the more publications tend to cover what caused the outage, and what changes have been made as a result.

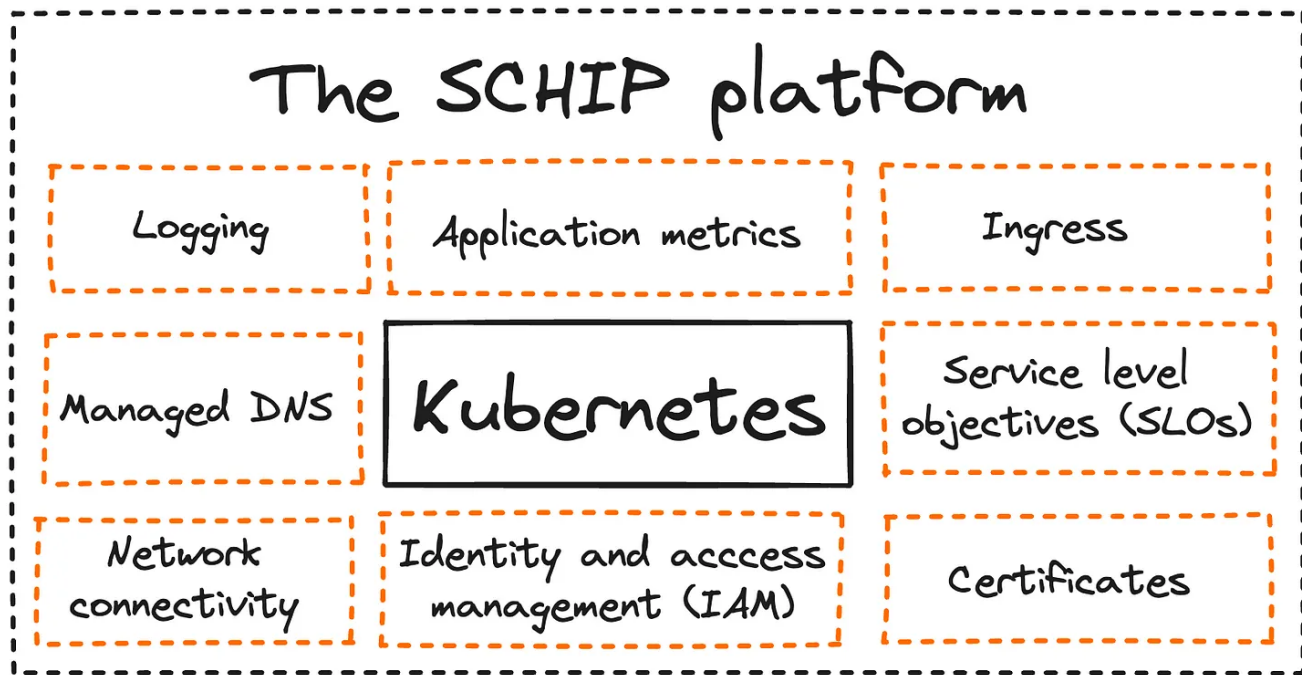
Writing a public-facing postmortem is a lot of effort, so it's usually only worth doing it for incidents that warrant this: either for widespread outages – as a way to communicate with all customers – or for outages with interesting learnings – as a way to circulate this knowledge across the tech community, and perhaps also help future hiring efforts.

With this: let's see some recent public incident reviews with interesting learnings.

2. Adevinta: a DNS murder mystery

Adevinta is a global classifieds company – and the company that bought eBay's classifieds business in 2021. The company operates sites for buying and selling vehicles, real estate, and finding jobs. A few of its brands include Gumtree (UK), Leboncoin (France), OLX and Zap (Brazil), DoneDeal (Ireland), Automobile.it (Italy) or Marktplaats (Netherlands).

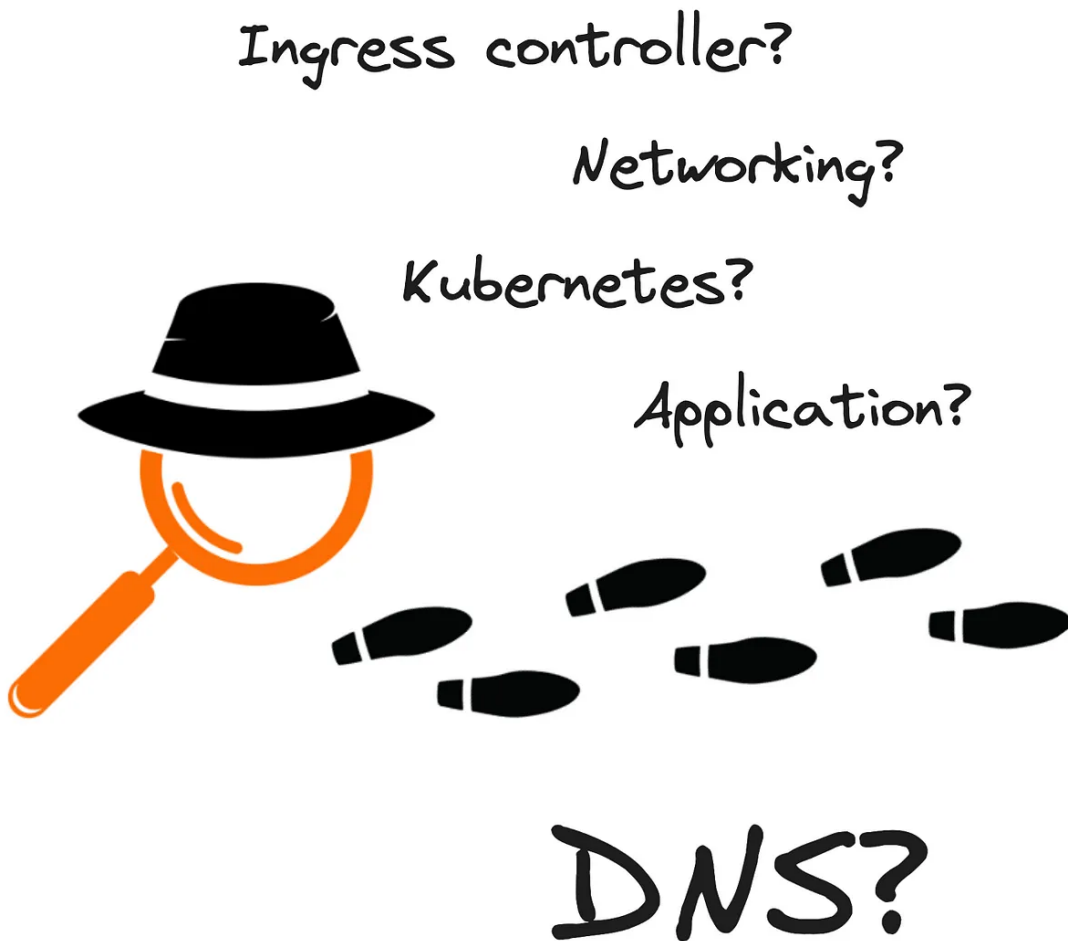
Given Adevinta serves dozens of local marketplaces, it is not really a surprise that the company operates an internal platform team, which offers shared infrastructure so product teams can build and run their own microservices. This platform is called SCHIP within Adevinta, and it adds services like observability, logging and service level objective (SLO) tracking, among others, on top of managing the container infrastructure with Kubernetes:



pragmaticengineer.com

The SCHIP platform is Adevinta's microservice platform. SCHIP adds additional capabilities on top of Kubernetes. Image data source: [Adevinta's engineering blog](#)

In May 2023, one of the teams on top of SCHIP noticed a spike of 5xx error rates. The team declared an incident, and started to investigate. This is where a long-running investigation that turned into what feels like a murder mystery started.



pragmaticengineer.com

Suspect #1: the ingress controller

Ingress refers to incoming network traffic, and is the opposite of egress, which is the outgoing traffic or data. Within SCHIP, the ingress controller is responsible for accepting traffic for the whole cluster of Kubernetes nodes. The team saw some signs of CPU throttling – a sign that some nodes could be overloaded and thus could perhaps drop packages – and so investigated this possibility first. But this investigation turned out to be a dead end.

But the errors still appeared, seemingly randomly, and for a short time, and even when the ingress controller had no issues.

Suspect #2: fluent bit and the networking layer

A new clue suggested the network was to blame for the error. This is thanks to the engineering team finding a log that said, "A fluent-bit agent stopped processing logs."

[Fluent Bit](#) is a popular open source log processor and forwarder that Adevinta uses. Fluent Bit is a common choice for cloud and containerized environments, and the project is part of the Cloud Native Computing Foundation (CNCF).

The thing that made Fluent Bit the new, prime, suspect, was how the 5xx error observed occurred on the same node as the Fluent Bit error. And the team speculated that there *must* be a networking issue connecting the two. But, after more investigation, this also turned out to be a dead end.

Suspect #3: *all* possible suspects

Tracking off the obvious leads led to nowhere, so the team decided to up the investigation one level, and created a dedicated investigation team with a few engineers. This team put together a dashboard to track all information they could think of to help track down the problem:

- All 5xx requests
- Network errors
- API usage metrics
- [Pod](#) metric changes
- DNS hit-and-miss
- DNS latency service level indicators (SLIs)

Did this dashboard help? Amusingly, it did not. From the incident review:

"The dashboard turned out to be more confusing because each combination of the indicators displayed made sense until they didn't."

Suspect #4: the application itself

Looking at all metrics, holistically, led to nowhere, so the team then started to explore the specific application that had issues. Until now, the team assumed the issue could

impact any application. But as they looked into the details, they noticed it only impacted one application.

The impacted application was a proxy server. This proxy server sat in front of microservices that made internal calls. And this proxy server used a Kubernetes name service resolution service. So this was a new suspect!

Suspect #5: the Kubernetes name service resolution service

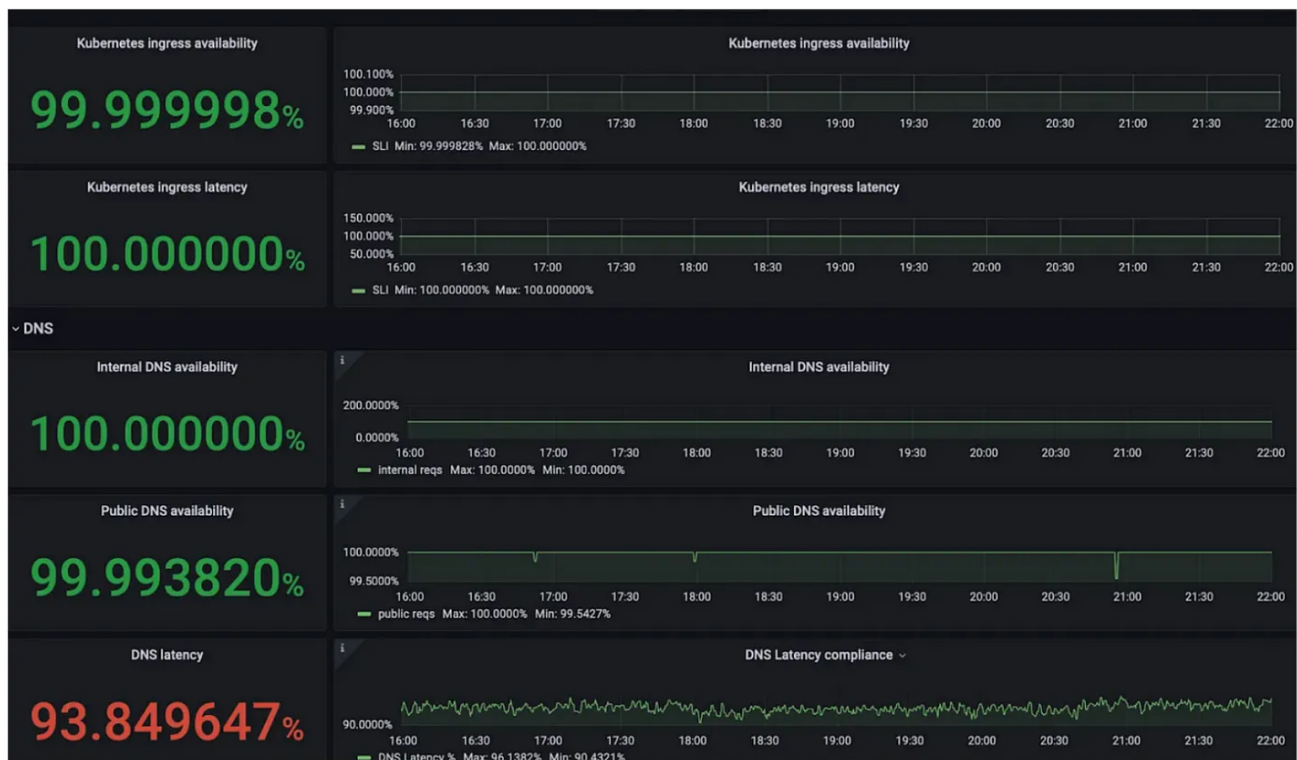
Kubernetes name service resolution service is a handy feature whereby you can call a service directly. For example, to invoke the "api" endpoint of an internal service called *search*, you can simply invoke the <http://search/api> URL. The name service resolution service takes care of mapping this URL to the right IP of the service.

The proxy service invoked internal services by using this simplified API scheme.

Also, remember suspect #2 – Fluent Bit – which had issues that could have been related to the network? Turns out Fluent Bit also used these simplified URLs, and it also had network-related issues.

Suspect #6: DNS

Resolving a hostname to the right IP doesn't happen reliably: that's related to domain name system (DNS) resolution. By this time, the engineering team was pretty certain that the issue *had* to be DNS-related. And more evidence was gathered as the team looked at the DNS latency service level objective (SLO) and saw that for the affected cluster, it was in the red:



Gotcha!

The DNS latency across clusters. The cluster that had reliability issues had latency that was well below the expected service level objective.

Image source: [Adevinta's engineering blog](#).

Out of other suspects, and with evidence pointing to a DNS error being the likely cause, was this non-deterministic outage caused by a DNS problem? Another detailed investigation followed, [which you can read here](#). In the end: yes, it was Adevinta's internal DNS implementation at fault, specifically the following:

1. **Too low of a concurrent DNS query** limit was set for the DNS cache, running on top of [dnsmasq](#);
2. **DNS cache misses.** The dnsmasq DNS cache service did not cache certain internal DNS requests;
3. **DNS request floods.** The team discovered their internal DNS services were flooded with hundreds of requests per second of absolute garbage (non-existent) DNS requests.

So in the end, the spike was the result of a mix of DNS cache issues, request flooding, and too low a DNS concurrent query limit set.

The biggest learnings

The investigation of the above issue took a whole month of a small team working on it. I asked site reliability engineer [Tanat Lokejaroenlarb](#) – who wrote up this review – and he told me:

"1. The importance of the investigation management

Without organizing the investigation effort, you could end up spending indefinite time on any issue, especially the uncommon ones. It's important to work on one theory at a time, focusing on proving it wrong and moving on to the next theory until you find the root cause.

2. The importance of SLIs (service level indicators)

Having flaky SLIs will reduce your trust in them. With flaky SLIs, eventually, these become just noise. Having reliable SLIs will provide you with solid indicators which will point you directly into the heart of the problems."

3. GitHub: when increasing reliability can cause a problem

I have previously praised GitHub for being transparent with its outages, [writing](#):

"I really admire the transparency of [GitHub's status page](#). While many vendors try to hide incidents, GitHub is highly transparent and reports incidents many vendors would normally not make public, such as degradation in performance in a subset of the system. Here is [one example](#), where git clone was slower than usual a week ago and so GitHub declared an incident. They then resolved the issue in 30 minutes."

The company keeps up with this good habit of being transparent about outages – even if those incidents are brief. On June 29, the service saw a 30-minute outage in some

parts of the US. The cause of this? GitHub exercising disaster recovery and finding an issue with failovers. From [their report](#):

"We have been working on building redundancy to an earlier single point of failure in our network architecture at a second Internet edge facility. This second Internet edge facility was completed in January and has been actively routing production traffic since then. Today we were performing a live failover test to validate that we could in fact use this second Internet edge facility if the primary were to fail.

Unfortunately, during this failover we inadvertently caused a production outage. During the test we exposed that the secondary site had a network pathing configuration issue that prevented it from properly functioning as the primary facility. This caused issues with Internet connectivity to GitHub, ultimately resulting in an outage."

Failover testing means moving an application from running in one location – for example, in one data center – to running in another one. Failover includes starting up the application in the new location, and redirecting traffic to this new location.

As GitHub was carrying out this exercise, it turned out that the failover did not work as intended.

There is a bit of irony when looking at the cause of this outage, because had GitHub *not* done an exercise to become more resilient, the service would not have suffered an outage. However, there's an upside to what happened. Because engineers knew that it was a failover test, they could mitigate it very quickly. It took two minutes from the first alert coming in to revert the change. It's safe to assume that under less controlled circumstances, starting the mitigation could have taken longer.

Also, it is *essential* that failovers work correctly: for the duration of an actual outage in one of the data facilities. While customers surely did not appreciate experiencing an outage, I'll personally take a more brief outage thanks to a vendor doing a failover test than a longer outage because the vendor cannot fail over.

In summary, GitHub confirmed they learned the lesson, and also apologized to its customers:

"This failover test helped expose the configuration issue, and we are addressing the gaps in both configuration and our failover testing which will help make GitHub more resilient. We recognize the severity of this outage and apologize for the impact it has to our customers."

Failover testing is increasingly important to validate proper reliability. If we've learned anything the past few months, it is how data centers can, and will, go down. The past three months, whole regions have had issues within AWS (us-east-1 [in June](#)), Google Cloud (europe-west-9 [in April](#)) and Azure (Western Europe [in July](#)). For all three outages, companies that were ready to fail over to another region could operate with little interruption. Those that did not have this ability – or those where the failover did not work – suffered a much longer outage.

If this GitHub outage signals anything, it is that GitHub is taking reliability visibly more seriously – in line [with the internal mandate at the company](#). With much more focus on reliability, I'd expect things to get a lot better, and then remain better, thanks to more resilient systems and practices being in place.

And as to the question on whether you should do failover testing, despite the fact that failovers introduce some risks? *Absolutely*. It is many times better to discover that your failover test has issues when your goal is to verify that failovers are working. If the failover doesn't work, you can revert to the original state. However, if you discover your failover is not working when you *need* to fail over, during an outage, this is when you'd really be in trouble.

Doing *proper* failover testing is not straightforward. It is easy enough to do a failover test when everything seems fine, and declare the failover a success. However, doing a failover testing under a "no-error condition" when all parts of the system work as expected can give a false sense of security.

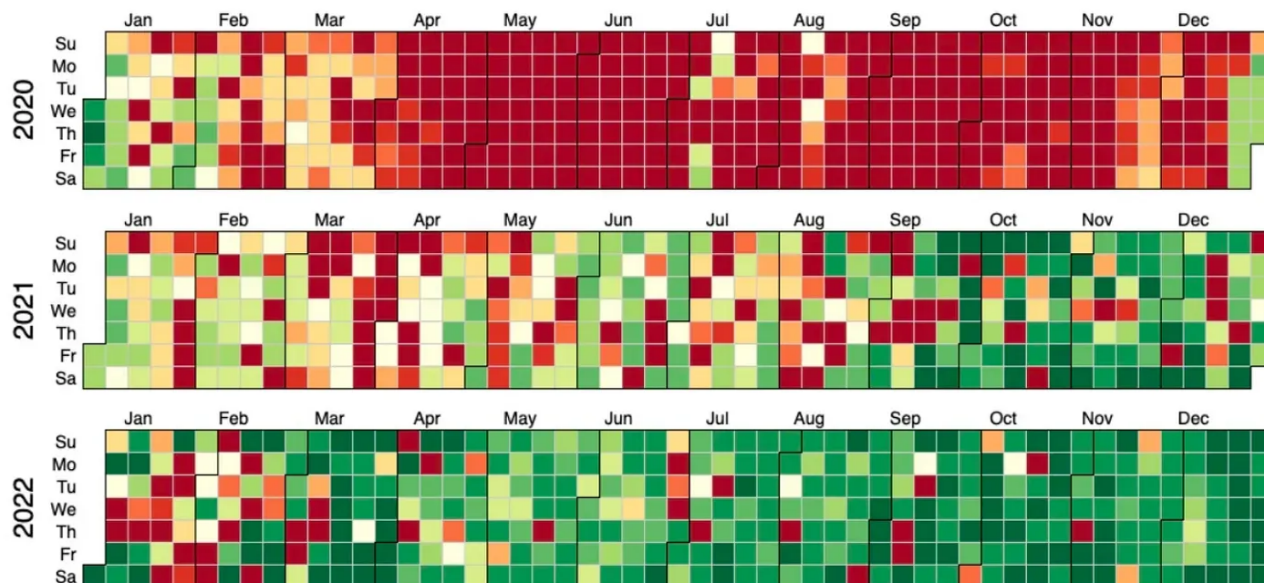
During an actual failover, you'd probably have a sudden issue come up in your main data center. This issue could mean that you cannot shut down apps or services properly, and this production failover then fails.

So when doing a failover exercise, aim to set up a scenario when the failover is instant, when apps and services fail over mid-state (and don't shut down properly), so you can validate that you can fail over under these more realistic conditions. Also: don't forget to do the failback exercise as well. It's not uncommon for the failover to work but the failback to have issues!

Of course, the more realistic the failover simulation, the higher the risk that the first time you do this failover you cause an outage. This is, unfortunately, a short-term risk to take for higher *overall* reliability.

4. Reddit: the difficult decision to try and fix on the spot, or to do a lengthy restore

One of Adevinta's learnings was the importance of SLOs. If Reddit is taking one thing seriously, it's SLOs: here is how the site has done in regards to their daily SLO targets over the past three years:



Reddit's service availability vs. their January 2023 SLO target. Source: [Reddit engineering](#).

That's a visible improvement trend across the site. But Reddit was down for close to five hours on March 14.

Reddit publishes their postmortems on Reddit, and they are not just educational reads, but also frequently entertaining. This is how the review starts:

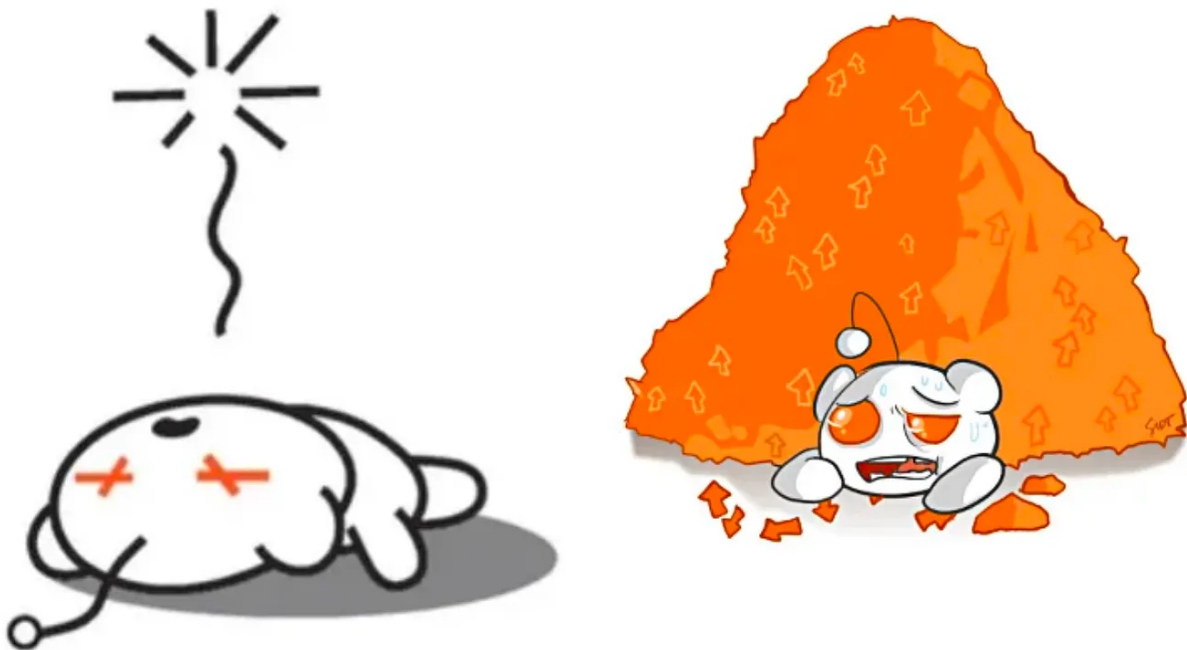
"It's funny in an ironic sort of way. As a team, we had just finished up an internal postmortem for a previous Kubernetes upgrade that had gone poorly; but only mildly, and for an entirely resolved cause. So we were kicking off another upgrade of the same cluster."

Reddit was upgrading from Kubernetes version 1.23 to 1.24. And this is what happened:

"This upgrade cycle was one of our team's big-ticket items this quarter, and one of the most important clusters in the company, the one running the Legacy part of our stack (affectionately referred to by the community as Old Reddit), was ready to be upgraded to the next version. The engineer doing the work kicked off the upgrade just after 19:00 UTC, and everything seemed fine, for about 2 minutes. Then?

Chaos."

YOU BROKE REDDIT.



When Reddit is down, the site displays cute error images like the one above.

The update process had gone wrong, and the Reddit team needed to decide how to proceed:

1. Do they debug the issue, live, and try to fix the issue? Like restarting services, attempting a forward fix?
2. Or do they restore the cluster from the backup, with restoration involving hours of guaranteed downtime?

The engineering team started with #1, and tossed around some ideas they had around resolving the issue. After an hour or two of no ideas working, they settled on starting the hours-long – and risky! – backup process.

Once the backup was complete, the team could breathe out and start their investigation on what *exactly* went wrong. The investigation is a mini mystery novel in itself, and, finally, the team found the root cause of the outage: how a Kubernetes node label naming was changed between releases.

Well, the label-naming change might have been the cause, but it was the symptom of something bigger:

“Really, that’s the proximate cause. The actual cause is more systemic, and a big part of what we’ve been unwinding for years: Inconsistency.

Nearly every critical Kubernetes cluster at Reddit is bespoke in one way or another. This is a natural consequence of organic growth, and one which has caused more outages than we can easily track over time. A big part of the Compute team’s charter has specifically been to unwind these choices and make our environment more homogeneous, and we’re actually getting there.”

The two biggest learnings I took from Reddit’s incident review were these:

- **Restoring in production is hard and scary.** The Reddit team were very open – and vulnerable – about just how stressful it was to do a full, production-level restore, when they had not done this exact restore before. You can simulate as many restorations as you want, but it’s not the same as bringing back all of production.

- **Inconsistent infrastructure can be a frequent source of outages.** Reddit grew rapidly over the years. Their infrastructure has become full of bespoke configurations – which is typical enough when teams are autonomous and move fast, unblocking themselves. The incident review reflects on both the challenges of such an in-situ legacy, and also outlines the plan on what Reddit is doing to bring more consistency to their infrastructure layer.

The postmortem is an unusually entertaining and informative read, and I highly recommend reading the original:

Takeaways

Thanks to Adevinta, GitHub and Reddit for publishing their postmortems for anyone to analyze. The biggest learnings I had from the respective incidents were these:

- **Good SLIs and SLOs can help track down outages faster.** In the case of Adevinta's goose hunt on the root cause of the outage: had the team paid more attention to the DNS latencies degrading in one of their clusters, they would have likely detected problems around this system earlier.
- **Disaster recovery procedures are risky, but you should still do them.** GitHub's outage came about after the engineering team did a failover exercise, preparing for disaster recovery scenarios. While downtime is never pleasant, the risk of downtime should not deter you from carrying out exercises like failovers and failbacks, or other disaster recovery scenarios. Of course, proceed with caution, and have the plan ready on how to roll back, just like GitHub was prepared for this.
- **Restoring a full backup in production is stressful, difficult and a hard call to make.** Reddit's incident review was very honest on how the team was hesitant to do a full rollback even as their Kubernetes upgrade had issues, because they hadn't done a restore like that *in production*. And even as they completed the restore, they had to improvise parts of it.
- **"Move fast with autonomous teams" often builds up infrastructure debt.** Reddit is a fast-moving scaleup where teams move fast, and it sounded like they had autonomy in infrastructure decisions. The wide range of infra configurations caused several outages, and the company is now paying down this "infrastructure

debt." This is not to say that autonomous teams moving fast is a bad thing, but it's a reminder that this approach introduces tradeoffs that could impact reliability and will eventually have to be paid down, often by dedicated teams.



150 Likes · 3 Restacks

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing